

Frontend -Assignment 1(Theory)

Q1. HTTP Request–Response Cycle

The HTTP Request–Response Cycle is the standard communication process between a client (browser) and a web server:

1. **DNS Lookup:** The browser converts the domain name into the server's IP address via DNS.
2. **TCP Connection:** The browser establishes a TCP/IP connection with the server (port 80 for HTTP, 443 for HTTPS).
3. **HTTP Request:** The browser sends a request with an HTTP method (GET, POST), headers, and optional body.
4. **Server Processing:** The server receives the request, runs backend logic, queries the database, and prepares the data.
5. **HTTP Response:** The server sends back a status code (e.g., 200 OK, 404 Not Found), headers, and the resource payload (HTML, CSS, JSON).
6. **Rendering:** The browser parses the received files and renders the webpage.

Q2. Web Technologies Overview

- **HTML:** Structural foundation of a webpage; defines content hierarchy using tags (headings, paragraphs, forms).
- **CSS:** Controls presentation and visual styling; manages colors, typography, Flexbox/Grid layouts, and responsive media queries.
- **JavaScript:** Adds dynamic behavior and interactivity; handles DOM manipulation, user events, and asynchronous API calls.
- **Bootstrap:** CSS framework for rapid UI development; provides a mobile-first 12-column grid and pre-built UI components.
- **jQuery:** JavaScript library that simplifies DOM selection, event handling, animations, and cross-browser AJAX calls.
- **React.js:** JavaScript library for building component-based UIs; uses a Virtual DOM for fast, dynamic rendering in single-page applications.

Q3. Client-Side vs. Server-Side Technologies

- **Client-Side:** Runs in the user's browser, handling the UI, layout, styling, and immediate user interactions. Examples: HTML, CSS, JavaScript, React.js.
- **Server-Side:** Runs on the backend server, handling business logic, database queries, authentication, and security. Examples: Node.js, Python, PHP, Java.

Key Differences:

- **Execution:** Client-side executes on the user's device; server-side executes on the remote host.
- **Visibility:** Client-side code is publicly visible in the browser; server-side code is hidden and secure.
- **Function:** Client-side focuses on user experience; server-side focuses on data processing and storage.

Q4. Block-Level Elements in HTML

Block-level elements always start on a new line and expand to occupy the full available width of their parent container.

Characteristics:

- Always begin on a new line.
- Occupy 100% of the parent width by default.
- Allow explicit width, height, margin, and padding styling.

Examples:

1. `<div>`: Generic container for grouping elements.
2. `<p>`: Paragraph container for text blocks.
3. `<h1>` to `<h6>`: Headings for document sections.

Q5. Semantic HTML Elements

Semantic elements clearly describe their meaning and purpose to both the browser and developer (e.g., `<article>` instead of `<div>`).

Importance:

- **Accessibility:** Helps screen readers navigate page structure for visually impaired users.
- **SEO:** Enables search engines to index and rank page content accurately based on hierarchy.

- **Maintainability:** Makes HTML cleaner, organized, and easier for developers to read.

Examples: <header>, <nav>, <article>, <section>, and <footer>.

Q6. HTML Table

HTML

```
<table border="1" cellpadding="8" cellspacing="0">
```

```
<thead>
```

```
<tr>
```

```
<th>Sr. No.</th>
```

```
<th>Name</th>
```

```
<th>Designation</th>
```

```
<th>Department</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
<tr>
```

```
<td>1</td>
```

```
<td>John</td>
```

```
<td>Manager</td>
```

```
<td>Sales</td>
```

```
</tr>
```

```
<tr>
```

```
<td>2</td>
```

```
<td>Smith</td>
```

```
<td>Sr. Manager</td>
```

```
<td>Marketing</td>
```

```
</tr>
```

```
<tr>
```

```
<td>3</td>
<td>Jane</td>
<td>Manager</td>
<td>HR</td>
</tr>
</tbody>
</table>
```

Q7. HTML Nested List

HTML

```
<ul>
<li>River
<ul>
<li>Ganga</li>
<li>Yamuna</li>
<li>Brahmaputra</li>
<li>Narmada</li>
</ul>
</li>
</ul>
```

Q8. Types of CSS

1. Inline CSS: Applied directly to an HTML element via the style attribute.

HTML

```
<p style="color: blue;">Inline text</p>
```

2. Internal CSS: Declared inside a <style> block within the <head> section of the HTML document.

HTML

```
<head>
<style>
```

```
body { background-color: #f4f4f4; }
```

```
</style>
```

```
</head>
```

3. External CSS: Written in a separate .css file and linked in the <head> tag.

HTML

```
<head>
```

```
<link rel="stylesheet" href="style.css">
```

```
</head>
```

Q9. HTML vs. CSS vs. JavaScript

- **HTML (Structure):** The skeleton of the webpage; defines the raw content and elements.
- **CSS (Presentation):** The styling and skin; controls visual layout, colors, and responsive design.
- **JavaScript (Behavior):** The muscles and logic; handles interactivity, form validation, and dynamic updates without page reload.

Q10. Short Answers

a. Role of Bootstrap: Bootstrap speeds up frontend development by providing a pre-built responsive grid system and ready-to-use styled UI components, ensuring consistent layout across mobile, tablet, and desktop screens.

b. What is jQuery and why it was widely used: jQuery is a lightweight JavaScript library. It became popular because it solved cross-browser compatibility issues, simplified verbose DOM manipulation into short syntax (`$('#id')`), and made AJAX calls easy.

c. What is React.js and what problem it solves: React.js is a component-based frontend library. It solves the performance and complexity issues of manual DOM manipulation by using a Virtual DOM, updating only the specific components that change rather than re-rendering the whole page.